

JavaScript this

[Back to JS page](#)

Table of Contents

- [JavaScript this](#)
 - [What is this?](#)
 - [this in an Object Method](#)
 - [Example 1](#)
 - [this in a Function \(Default\)](#)
 - [Example 2](#)
 - [this Alone](#)
 - [Example 3](#)
 - [this in Event Handlers](#)
 - [Example 4](#)
 - [this in Arrow Functions](#)
 - [Example 5](#)
 - [Document](#)
 - [Reference](#)

What is this?

In JavaScript, the `this` keyword refers to an **object**.

Which object depends on how `this` is being invoked (used or called).

The `this` keyword refers to different objects depending on how it is used:

- In an object method, `this` refers to the **object**
- Alone, `this` refers to the **global object**
- In a function, `this` refers to the **global object**
- In a function (in strict mode), `this` is `undefined`
- In an event, `this` refers to the **element** that received the event
- Methods like `call()`, `apply()`, and `bind()` can refer `this` to **any object**

this in an Object Method

When used in an object method, `this` refers to the object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript this</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>this in an Object Method</h4>
<p id="demo"></p>

<script>
const person = {
  firstName: "John",
  lastName: "Doe",
  id: 5566,
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};

document.getElementById("demo").innerHTML = person.fullName();
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

this in a Function (Default)

In a regular function, `this` refers to the global object (window in a browser):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript this</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>this in a Function (Default)</h4>
<p id="demo"></p>

<script>
function myFunction() {
    return this;
}

document.getElementById("demo").innerHTML = myFunction();
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

this Alone

When used alone, `this` refers to the global object (window in a browser):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript this</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>this Alone</h4>
<p id="demo"></p>

<script>
let x = this;
document.getElementById("demo").innerHTML = "this alone: " + x;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

this in Event Handlers

In HTML event handlers, `this` refers to the HTML element that received the event:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript this</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>this in Event Handlers</h4>
<button onclick="this.style.display='none'">Click to remove me!</button>
<br><br>
<button id="myBtn">Click to hide (using JS)</button>

<script>
document.getElementById("myBtn").addEventListener("click", function() {
  this.style.display = 'none';
});
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

this in Arrow Functions

Arrow functions do NOT have their own `this`. Instead, they inherit `this` from the surrounding scope.

This makes arrow functions useful in callbacks and methods where you want to preserve the context:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript this</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>this in Arrow Functions</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
// Regular function - has its own this
const obj1 = {
  name: "Object 1",
  greet: function() {
    return "Hello from " + this.name;
  }
};

// Arrow function - inherits this from surrounding scope
const obj2 = {
  name: "Object 2",
  greet: () => {
    return "Hello from " + this.name; // this is NOT obj2
  }
};

document.getElementById("demo1").innerHTML = obj1.greet();
document.getElementById("demo2").innerHTML = obj2.greet() + " (arrow function - this is window)"
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript this](#)